

Distributed Web Crawling and Parallel PageRank Computation

Milestone 3 Report

Incremental Updates and System Evaluation

Course: Parallel and Distributed Computing

Group Members

Abdullah Irfan 29266

Ameer Hamza 28308

Muhammad Khizer 28991

Rameez Hoda 29057

Note on AI Assistance: This report and the corresponding code were refined, enhanced, and made grammatically correct using Claude Sonnet 4.5 (Anthropic).

April 29, 2026

Contents

1	Project Overview	3
1.1	What the Project is About	3
1.2	Recap of Milestone 1	3
1.3	Recap of Milestone 2	3
1.4	What Milestone 3 is About	4
1.5	Note on Graph Size and Reproducibility	4
2	System Architecture	5
2.1	Pipeline Overview	5
2.2	File Structure	5
2.3	Data Flow	5
2.4	Technology Stack	6
3	Detailed Module Descriptions	6
3.1	m3_config.py – Central Configuration	6
3.2	instrumentation.py – Runtime Statistics Collector	7
3.3	partitioning_strategies.py – Pluggable Partitioning	7
3.4	incremental_crawler.py – Snapshot Growth	8
3.5	incremental_pagerank.py – Update Strategies	9
3.6	evaluation_runner.py – Run-Matrix Orchestrator	10
3.7	plot_results.py – Figure Generation	10
3.8	m3_main.py – CLI Entry Point	10
4	Incremental Crawling Methodology	10
5	Incremental PageRank Algorithm	11
5.1	Base PageRank Update	11
5.2	Strategy 1: Full Recomputation	11
5.3	Strategy 2: Warm-Start	12
5.4	Strategy 3: Localised Update	12
5.5	Edge-Balanced Partitioning Detail	12
6	Experimental Setup	12
6.1	Hardware and Software	12
6.2	Run Matrix	13
7	Experimental Results	13
7.1	Group A – Sequential vs Parallel PageRank	13
7.2	Group B – Partitioning Strategy Comparison	15
7.3	Group C – Static vs Incremental Computation	18
7.4	Memory Timeline (Cross-Cutting)	21
8	Discussion	22
8.1	Sequential vs Parallel: the Same Crossover Point	22
8.2	Edge-Balanced Partitioning Pays Off Only for Long Runs	22
8.3	Warm-Start is the Practical Default	22
8.4	Localised Updates: a Cautionary Tale	23
8.5	Communication Volume Insensitive to Strategy	23
8.6	Limitations	23

9 Milestone 3 Deliverables Checklist	23
10 Work Done Across All Three Milestones	24
10.1 Milestone 1 – Completed (March 2026)	24
10.2 Milestone 2 – Completed (April 2026)	24
10.3 Milestone 3 – Completed (April 2026)	25
11 Conclusion	25
References	26

1 Project Overview

1.1 What the Project is About

This project is about building a system that can crawl a large number of web pages, build a graph from those pages, and then run the PageRank algorithm on that graph in parallel. The main idea is not just to get it working, but to do it in a way that uses multiple CPU cores or multiple machines at the same time, which is the core of Parallel and Distributed Computing.

The web can be thought of as a directed graph where each page is a node and every hyperlink from one page to another is a directed edge. PageRank is an algorithm originally developed by Google that assigns an importance score to every node based on how many other important nodes link to it. Computing PageRank on a large graph with millions of nodes takes a long time if done sequentially, which is why parallelism matters here.

The project is divided into three milestones that build on each other:

- **Milestone 1** builds the web graph by crawling pages and extracting links in parallel.
- **Milestone 2** takes that graph and runs the PageRank algorithm in parallel across multiple workers.
- **Milestone 3** (this report) extends the system to handle new pages being added *after* PageRank has already been computed, introduces incremental update strategies, and produces a comprehensive system-level evaluation across multiple axes.

The system is implemented using Ray, a Python-based parallel runtime that makes it easy to run functions on multiple CPU cores and manage shared state between them.

1.2 Recap of Milestone 1

In Milestone 1 we implemented a parallel web-crawling pipeline that:

- Used Ray workers to concurrently fetch and parse web pages from CommonCrawl WARC archives, demonstrating task parallelism.
- Extracted hyperlinks to construct a directed graph where nodes represent pages and edges represent hyperlinks.
- Employed shared-state coordination via a **GraphActor** to accumulate edges across all workers without race conditions.
- Applied configurable depth and worker constraints to produce deterministic, reproducible graph workloads.
- Serialised the resulting NetworkX **DiGraph** as a pickle file (**web_graph.pkl**) for downstream consumption.

Two graph configurations were produced:

- Config 1 (Small): 199 source pages $\rightarrow$ 15,657 nodes, 25,576 edges
- Config 3 (Medium): 496 source pages $\rightarrow$ 37,074 nodes, 67,667 edges

Config 3 serves as the baseline (v_0 **snapshot**) for the incremental experiments in Milestone 3.

1.3 Recap of Milestone 2

Milestone 2 took the directed graph produced by Milestone 1 and applied the PageRank algorithm to compute importance scores for every node. The key contributions were:

- A custom iterative PageRank implementation following the standard formula with explicit dangling-node handling.
- Two parallel execution strategies built on Ray: *centralised aggregation* (driver-side merge) and *distributed reduction* (Ray actor-based accumulation).
- Range-based graph partitioning with one partition per worker.
- Convergence-based and fixed-iteration termination modes.
- A 14-test experimental suite covering worker scaling, strategy comparison, termination policy, graph-size sensitivity, and damping factor sensitivity.
- Comprehensive metrics: speedup, efficiency, communication volume, load imbalance, Gini coefficient, and Amdahl’s Law analysis.

The headline finding from Milestone 2 was that on graphs of this size ($\sim 37,000$ nodes), Ray’s task-dispatch and serialisation overhead exceeds the actual computation time per iteration, producing *sub-linear speedup*. This was a core PDC insight: parallelisation has a break-even point that depends on the ratio of computation to framework overhead. Milestone 3 builds on this baseline, holding the parallel infrastructure constant while introducing dynamic graph growth and incremental update strategies.

1.4 What Milestone 3 is About

Milestone 3 is the system-level evaluation phase of the project. Its goal is to study how dynamic data affects parallel algorithms and to evaluate system behaviour under changing workloads. The key objectives are:

1. **Extend the crawler** to introduce new pages after the initial PageRank has converged, producing a sequence of graph snapshots (v_0, v_1, v_2, v_3) of increasing size.
2. **Update PageRank scores incrementally** via three distinct strategies: full recomputation (cold restart), warm-start (reuse previous scores as initial vector), and localised update (recompute only the dirty subgraph affected by new edges).
3. **Analyse the trade-off between recomputation cost and convergence accuracy** for each strategy, on graphs ranging from 37K to 100K nodes.
4. **Instrument the system** to collect detailed runtime statistics including per-task duration, communication volume, peak memory usage, dirty-set evolution, and load imbalance.
5. **Compare partitioning strategies** (range, hash, edge-balanced) to study how partition quality affects parallel PageRank performance.
6. **Produce a structured evaluation** comparing sequential vs. parallel PageRank, the three partitioning strategies, and static vs. incremental computation.

PDC focus: dynamic workloads, recomputation strategies, performance analysis under changing conditions.

1.5 Note on Graph Size and Reproducibility

As noted in the Milestone 2 report, the CommonCrawl CDX index server experienced prolonged downtime during initial development. To prevent this from blocking Milestone 3, every CDX query and every fetched WARC record is now cached on disk under `Milestone 3/cache/`. The first crawl of each new batch hits the network; every subsequent crawl reads from the cache, making the entire pipeline deterministic and reproducible even when CommonCrawl is unavailable. This change preserves the original “CommonCrawl-only” design philosophy while making the experiments re-runnable on any machine with a populated cache.

The graphs grow from 37,074 nodes (v_0) to 99,803 nodes (v_3) across the three incremental crawls. While still smaller than a production web-graph, this range is large enough to expose the trade-offs between full recomputation and incremental update strategies, which is the core PDC objective of Milestone 3.

2 System Architecture

2.1 Pipeline Overview

The Milestone 3 system extends the architecture of Milestones 1 and 2 with three new layers:

1. An **incremental crawler** that grows the graph snapshot by snapshot, persisting each version and its delta to disk.
2. An **instrumentation layer** that records detailed per-iteration runtime statistics for every PageRank run.
3. An **evaluation harness** that orchestrates the three experiment groups (A: sequential vs. parallel, B: partitioning strategies, C: static vs. incremental) and aggregates results.

The PageRank computational core itself is reused from Milestone 2 but extended to support warm-start initialisation, localised dirty-set recomputation, and pluggable partitioning strategies.

2.2 File Structure

```
Milestone 3/
|-- m3_config.py           # Central configuration dataclass
|-- instrumentation.py     # RuntimeStats collector
|-- partitioning_strategies.py # range / hash / edge_balanced
|-- incremental_crawler.py  # Snapshot growth + caching
|-- incremental_pagerank.py  # full / warm / localised
|-- evaluation_runner.py    # Groups A, B, C orchestrator
|-- plot_results.py         # 13 publication figures
|-- m3_main.py             # Single CLI entry point
|-- requirements_m3.txt
|-- How to run.txt
|-- cache/                 # On-disk cache of CDX + WARC fetches
|-- snapshots/             # graph_v{0,1,2,3}.pkl + deltas
|-- output/
|   |-- groupA/            # Sequential vs parallel results
|   |-- groupB/            # Partitioning strategy results
|   |-- groupC/            # Static vs incremental results
|   '-- figures/           # 13 .png + 13 .pdf
'-- report/
    |-- Milestone3_Report.tex
    '-- figures/
```

Listing 1: Milestone 3 directory layout (relative to repo root)

2.3 Data Flow

1. `m3_main.py crawl-incremental` drives the `incremental_crawler`, which fetches new CDX pages from CommonCrawl (or from cache), extracts links via the Milestone 1 parser, and produces the next snapshot v_{i+1} along with a delta file describing exactly which nodes and edges were added.
2. `m3_main.py evaluate` loads the snapshots, executes the Group A/B/C run matrix via `evaluation_runner`, and writes structured metrics under `output/group{A,B,C}/`.
3. `m3_main.py plot` reads the saved metrics and produces the 13 figures in `output/figures/`.

2.4 Technology Stack

Table 1: Technology stack for Milestone 3

Component	Technology
Language	Python 3.12
Parallel runtime	Ray $\geq 2.9.0$
Graph library	NetworkX $\geq 3.2.0$
Numerical backend	NumPy $\geq 1.26.0$
Memory profiling	psutil
Plotting	matplotlib
HTTP / WARC ingest	requests, warcio (Milestone 1 reuse)
HTML parsing	BeautifulSoup4 + lxml

3 Detailed Module Descriptions

3.1 m3_config.py – Central Configuration

3.1.1 Purpose and Design

This module defines the `M3Config` dataclass that centralises every tunable parameter for Milestone 3. It mirrors the design pattern of `config.py` (Milestone 1) and `pagerank_config.py` (Milestone 2): no magic numbers in algorithm files, every experiment fully reproducible from its configuration alone.

3.1.2 Implementation

The dataclass groups parameters into five sections:

- **Crawl settings:** CDX query parameters reused from Milestone 1, the per-step page budget, the snapshot directory, and the cache directory.
- **PageRank parameters:** damping factor ($d = 0.85$), convergence threshold ($\epsilon = 10^{-6}$), maximum iterations (100), and termination mode.
- **Parallelism settings:** default number of Ray workers (4), default partitioning strategy, and Ray address (local by default).
- **Strategy settings:** dirty-set k -hop expansion factor for localised updates, warm-start enable flag, and reference-strategy selector (`full` by default) used as the ground-truth for accuracy comparisons.
- **Output settings:** per-group output subdirectories, metrics save-format flags, and figure DPI.

3.1.3 Design Decisions

Defaulting `num_partitions = num_workers` produces one partition per worker, which is the optimal mapping for range and hash partitioning. The dirty-set expansion factor k is exposed as a configuration parameter rather than hardcoded so that the cost-vs-accuracy trade-off of localised updates can be tuned and studied.

3.2 instrumentation.py – Runtime Statistics Collector

3.2.1 Purpose and Design

The instrumentation module is the data-collection backbone of Milestone 3. It provides a single `RuntimeStats` class that every PageRank run uses to record detailed per-iteration metrics. The collected statistics are then serialised to JSON and CSV for downstream analysis and plotting.

3.2.2 Implementation

The module exposes three abstractions:

- `IterationRecord` dataclass: captures, for one PageRank iteration, the wall-clock time, per-task durations (min/max/avg/std), driver memory, peak worker RSS, communication volume in bytes, dirty-set size, and the convergence delta.
- `Stopwatch` context manager: thin wrapper around `time.perf_counter()` used to time both the whole run and individual phases. Returns elapsed seconds when the `with` block exits.
- `RuntimeStats` class: the main collector. Provides `record_iteration()` for per-iteration logging, `record_metadata()` for run-level metadata (graph size, strategy, worker count, etc.), and `save()` which writes `run_metrics.json` (full structured record) and `iterations.csv` (one row per iteration).

3.2.3 Design Decisions

Capturing both per-task min/max times and peak worker RSS allows the load-imbalance and memory analyses in Section 7. Communication volume is computed analytically as $N \times 8 \times \text{iterations}$ bytes (rank vector size $\times$ number of synchronisation rounds), matching the methodology introduced in Milestone 2.

3.3 partitioning_strategies.py – Pluggable Partitioning

3.3.1 Purpose and Design

Milestone 2 used range partitioning exclusively. Milestone 3 introduces two additional strategies and a unified dispatcher so that the parallel PageRank core can be invoked with any of them.

3.3.2 Implementation

Three partitioning functions are provided:

Range partitioning. Splits the node ID range $[0, N)$ into k contiguous chunks of approximately equal size. This is the Milestone 2 baseline. It produces perfectly balanced partition counts but does not balance work, since per-node computation depends on in-degree.

Hash partitioning. Assigns node u to partition $h(u) \bmod k$ where h is a stable 64-bit hash. This destroys the locality of the original node-ID ordering and tends to spread high-in-degree nodes uniformly across partitions, reducing load imbalance compared to range.

Edge-balanced partitioning. Estimates the per-node work as $w_u = |\text{In}(u)| + 1$ (the “+1” prevents zero-in-degree nodes from being treated as free, since they still pay the loop overhead). Nodes are sorted in descending order of w_u and assigned greedily to the partition with the currently smallest total work, an instance of the Longest-Processing-Time-first (LPT) heuristic. This produces near-equal total work per partition.

Dispatcher and diagnostics. A `get_partitions()` function selects the strategy by name. A `partition_diagnostics` helper computes and reports per-partition node counts, total in-degree, and the resulting load-imbalance metric $(\max_p W_p - \min_p W_p) / \max_p W_p$ where $W_p = \sum_{u \in p} w_u$.

3.3.3 Design Decisions

LPT is asymptotically a 4/3-approximation to optimal makespan scheduling, which is sufficient given that the bottleneck on this graph size is overhead rather than load imbalance. The strategies are deliberately pure-Python and do not require external libraries (e.g., METIS), preserving the project’s portability constraint that the code “run on any other PC unless there are specific constraints”.

3.4 incremental_crawler.py – Snapshot Growth

3.4.1 Purpose and Design

This is the dynamic-data layer of Milestone 3. It extends the Milestone 1 crawler so that the graph can grow snapshot by snapshot, with each snapshot persisted alongside a delta file describing exactly what was added. A persistent on-disk cache makes re-runs deterministic and network-free.

3.4.2 Implementation

The module provides the following key functions:

Cache layer. `_cache_key()`, `_cache_path()`, `_read_cache()`, and `_write_cache()` hash the URL and fetch parameters into a stable filename, then read or write a pickled (`headers`, `html`) tuple under `cache/`. The CDX index itself is also cached as a JSON list so that the seed selection is deterministic.

Cache-aware Ray task. A new `@ray.remote` task `cached_process_batch` replaces Milestone 1’s `process_batch`. For each URL in its batch it first checks the cache; on a hit it returns the cached HTML; on a miss it fetches the WARC payload, writes it to the cache, and returns. Extracted edges are streamed to the shared `GraphActor`.

Snapshot and delta IO. `save_snapshot()` pickles the new `DiGraph` as `graph_v{i}.pkl` and also writes a JSON node list and a CSV edge list for human inspection. `compute_delta()` compares two snapshots and produces a `delta_v{i}_v{i+1}.json` file containing the new source URLs, the added node set, the added edge list, and counts.

Actor seeding. `_seed_actor()` pre-populates the `GraphActor` with the previous snapshot’s edges before the new crawl begins, so that the incremental run produces a strict superset of the previous snapshot rather than a fresh graph.

Orchestration. `run_incremental_step()` wires the above together to produce one new snapshot. `ensure_v0_snapshot()` copies the Milestone 1 Config 3 graph into `snapshots/graph_v0.pkl` as the starting point. `run_full_growth_plan()` drives all three steps ($v_0 \rightarrow v_1 \rightarrow v_2 \rightarrow v_3$).

3.4.3 Design Decisions

Caching is keyed on the URL plus the Milestone 1 fetch parameters, so parameter changes invalidate the cache automatically. The `PYTHONPATH` of the driver is explicitly propagated to Ray worker subprocesses (Ray does not inherit `sys.path` modifications from the driver), allowing remote tasks to import Milestone 1 modules without restructuring those modules.

3.5 incremental_pagerank.py – Update Strategies

3.5.1 Purpose and Design

This module implements the three incremental PageRank update strategies that are central to the Milestone 3 evaluation. The parallel inner loop is shared across all strategies; the strategies differ only in how the initial rank vector is constructed and which nodes are included in the update set.

3.5.2 Implementation

The module exposes:

Graph preparation. `load_graph_for_pagerank()` loads a snapshot pickle and converts it into integer-indexed in-link arrays, out-degree arrays, and a dangling-node set, mirroring the `graph_loader.py` contract from Milestone 2. `build_out_links()` additionally builds an out-link adjacency list which is needed by the dirty-set expansion.

Initial rank vectors. `initial_ranks_uniform(N)` returns a vector of $1/N$ values. `initial_ranks_warm(prev_scores, current_url_to_idx)` copies the previous score for every URL that existed in the previous snapshot, assigns $1/N$ to genuinely new URLs, and rescales so that the sum equals 1.

Dirty-set computation. `compute_dirty_set(prev_graph, new_graph, k=2)` computes the set of nodes whose ranks may change due to the new edges. The seed is the union of (a) all sources of new edges, (b) all destinations of new edges, (c) all nodes that gained or lost an in-edge. The seed is then expanded by k levels of out-link BFS, because a rank change on node u propagates forward to its out-neighbours over PageRank iterations.

Parallel inner loop. `_run_parallel_pagerank()` is the strategy-agnostic core. It accepts the in-link/out-degree arrays, the partition list, the initial rank vector, an optional dirty mask, the partition strategy, the worker count, and the convergence parameters. Each iteration dispatches one Ray task per partition, performs a barrier via `ray.get()`, merges results, computes the convergence delta, and records an `IterationRecord` via the instrumentation layer.

Three public strategies.

- `run_full_recomputation()` – cold restart from uniform initial ranks. This is the ground-truth reference against which warm-start and localised are compared.
- `run_warm_start()` – starts from the previous snapshot’s scores (extended with $1/N$ for new URLs) but updates *all* nodes every iteration.
- `run_localised_update()` – starts from warm initial ranks but only updates nodes inside the dirty set; the rest keep their warm values.

Sequential baseline and accuracy comparison. `run_sequential()` is the single-threaded PageRank used as the speedup baseline in Group A. `compare_to_reference()` computes the maximum absolute difference, L_1 difference, and L_2 difference between two score vectors and is used in Group C accuracy reporting.

3.6 `evaluation_runner.py` – Run-Matrix Orchestrator

This module drives the full Group A / B / C experimental matrix from a single Python entry point. For each group it constructs the appropriate configuration variants, calls into the relevant PageRank strategy, and aggregates per-run metrics into a per-group summary JSON file.

Group A runs the sequential baseline followed by parallel range-partitioned PageRank with worker counts $\{1, 2, 4\}$ on the v_0 snapshot. Speedup and efficiency are derived from the sequential baseline.

Group B runs parallel PageRank with 4 workers on v_0 using each of the three partitioning strategies, recording load imbalance, communication volume, and peak per-worker RSS.

Group C runs the three update strategies (full / warm / localised) for each of the three transitions ($v_0 \rightarrow v_1$, $v_1 \rightarrow v_2$, $v_2 \rightarrow v_3$), with full recomputation acting as the per-transition reference for accuracy comparison.

3.7 `plot_results.py` – Figure Generation

This module reads the per-group summary JSON files plus the underlying per-run `run_metrics.json` files and produces 13 publication figures in `output/figures/`, each saved as both PNG and PDF. The figures cover the three experimental groups as well as a cross-cutting memory timeline. All figures are rendered at 150 DPI with matplotlib’s default style for consistency with the Milestone 2 report.

3.8 `m3_main.py` – CLI Entry Point

A single argparse-based CLI with four subcommands:

- **crawl-incremental** – runs the snapshot growth plan ($v_0 \rightarrow v_3$).
- **pagerank** – runs a single PageRank invocation with a user-selected strategy, partitioning, and worker count, useful for smoke-testing.
- **evaluate** – runs Groups A, B, and/or C.
- **plot** – regenerates all figures from saved metrics.

This single-entry-point design mirrors `main.py` (Milestone 1) and `main_pagerank.py` (Milestone 2), keeping the developer experience consistent across the three milestones.

4 Incremental Crawling Methodology

The Milestone 3 crawler is conceptually a wrapper around Milestone 1 that introduces (i) snapshot versioning, (ii) delta computation, and (iii) caching. The flow for one incremental step is:

1. **Load previous snapshot.** The previous DiGraph pickle is loaded and used to seed the `GraphActor`.
2. **Discover new seed URLs.** A new batch of CDX records is fetched (or read from cache) and filtered to exclude URLs already crawled.
3. **Run the parallel crawl.** New URLs are sharded into batches and processed by Ray workers using the cache-aware task. Each worker writes extracted edges to the shared `GraphActor`.
4. **Materialise the new snapshot.** The actor’s accumulated edge list is converted to a DiGraph and serialised as `graph_v{i+1}.pkl`.
5. **Compute the delta.** The new and old snapshots are diffed, producing a JSON file listing exactly which nodes and edges were added.

The starting snapshot v_0 is the Milestone 1 Config 3 graph (37,074 nodes, 67,667 edges). Three incremental steps were executed, each adding 100, 200, and 500 new pages respectively. Table 2 summarises the resulting snapshot sequence.

Table 2: Incremental snapshot sequence

Snapshot	Source pages	Nodes	Edges	Crawl time (s)
v_0 (M1 Config 3)	496	37,074	67,667	—
v_1	596	45,830	82,550	109.4
v_2	791	61,720	110,464	161.9
v_3	1,288	99,803	178,696	396.5

The corresponding delta sizes are listed in Table 3. The delta grows superlinearly in the number of new source pages because each new page introduces edges to existing nodes and to brand-new dangling nodes that were not previously referenced.

Table 3: Per-step delta summary

Transition	Added nodes	Added edges
$v_0 \rightarrow v_1$	8,756	14,883
$v_1 \rightarrow v_2$	15,890	27,914
$v_2 \rightarrow v_3$	38,083	68,232

5 Incremental PageRank Algorithm

5.1 Base PageRank Update

All three Milestone 3 strategies share the same per-iteration update rule, inherited from Milestone 2. For each node u at iteration t :

$$r_u^{(t+1)} = \frac{1-d}{N} + \frac{d \cdot S_{\text{dangling}}^{(t)}}{N} + d \sum_{v \in \text{In}(u)} \frac{r_v^{(t)}}{\text{out}(v)} \quad (1)$$

where $d = 0.85$ is the damping factor, N is the total number of nodes, $S_{\text{dangling}}^{(t)} = \sum_{j \in D} r_j^{(t)}$, and D is the set of dangling nodes (those with out-degree zero).

The convergence criterion is the L_1 norm of the rank-change vector:

$$\delta^{(t)} = \sum_u \left| r_u^{(t+1)} - r_u^{(t)} \right| \quad (2)$$

Iteration stops when $\delta^{(t)} < \epsilon$ (default 10^{-6}).

5.2 Strategy 1: Full Recomputation

A cold restart on the new snapshot. The initial rank vector is uniform: $r_u^{(0)} = 1/N$ for every node u . All nodes are updated every iteration. This is the most accurate strategy and serves as the ground-truth reference; its only drawback is that it discards all the work done in computing the previous snapshot’s scores.

5.3 Strategy 2: Warm-Start

The intuition is that, after a small graph delta, the new PageRank vector should be close to the old one. Starting iteration from the old scores should therefore reduce the number of iterations needed to converge.

The warm initial vector is constructed as follows: for each node u in the new graph, $r_u^{(0)} = \tilde{r}_u$ if the URL associated with u existed in the previous snapshot (where $\tilde{r}_u$ is its previous PageRank score); otherwise $r_u^{(0)} = 1/N$. The vector is finally rescaled so that $\sum_u r_u^{(0)} = 1$.

The standard parallel update is then applied iteratively over *all* nodes (not just the dirty set), using Equation 1. Convergence still terminates on $\delta < \epsilon$.

5.4 Strategy 3: Localised Update

The most aggressive incremental strategy. It computes a *dirty set* D_{dirty} of nodes whose ranks may have been affected by the delta and updates only those, leaving other nodes at their warm-start values.

Dirty-set seed. The initial seed is the union of:

- All sources of newly-added edges.
- All destinations of newly-added edges.
- Every node whose in-link set changed (gained or lost an in-edge).

k -hop expansion. Because rank changes propagate forward along out-edges in PageRank, the seed is expanded by k levels of out-link BFS. Empirically $k = 2$ provided a useful trade-off between correctness (more nodes in the dirty set $\Rightarrow$ more accurate results) and cost (fewer nodes $\Rightarrow$ faster updates).

Update loop. The iteration is identical to the warm-start update except that nodes outside D_{dirty} are not recomputed: their rank value is held fixed at the warm initial value. The convergence criterion is restricted to nodes inside the dirty set ($\delta_{\text{dirty}}^{(t)} = \sum_{u \in D_{\text{dirty}}} |r_u^{(t+1)} - r_u^{(t)}|$).

5.5 Edge-Balanced Partitioning Detail

Edge-balanced partitioning (Group B) is implemented by greedy LPT:

1. Compute $w_u = |\text{In}(u)| + 1$ for every node u .
2. Sort nodes by descending w_u .
3. Maintain k partition-load counters $\{W_1, \dots, W_k\}$ initialised to zero.
4. For each node u in sorted order, assign it to $\arg \min_p W_p$ and increment that W_p by w_u .

This produces near-equal total in-degree per partition, which directly proxies the per-partition computation cost in Equation 1.

6 Experimental Setup

6.1 Hardware and Software

- **Machine:** Local development workstation, Windows 10, 8-core x86_64 CPU, 16 GB RAM.
- **Python:** 3.12 in an isolated virtual environment.
- **Ray:** 2.9+ in local mode (single-machine multi-process).

- **All measurements:** wall-clock time via `time.perf_counter()`, memory via `psutil`.

6.2 Run Matrix

The full evaluation comprises three groups of runs. Each group is self-contained: it can be re-run independently and produces its own JSON summary plus per-run metrics.

Table 4: Group A – Sequential vs Parallel (snapshot v_0 , range partitioning)

Run	Strategy	Workers
A1	Sequential	1
A2	Parallel (full recomputation)	1
A3	Parallel (full recomputation)	2
A4	Parallel (full recomputation)	4

Table 5: Group B – Partitioning Strategies (snapshot v_0 , 4 workers, full recomputation)

Run	Partition strategy
B1	range
B2	hash
B3	edge_balanced

Table 6: Group C – Static vs Incremental (4 workers, range partitioning)

Run	Transition	Strategy
C1, C2, C3	$v_0 \rightarrow v_1$	full / warm / localised
C4, C5, C6	$v_1 \rightarrow v_2$	full / warm / localised
C7, C8, C9	$v_2 \rightarrow v_3$	full / warm / localised

All runs use convergence-based termination ($\epsilon = 10^{-6}$, max 100 iterations).

7 Experimental Results

7.1 Group A – Sequential vs Parallel PageRank

Table 7 reports the wall-clock times, observed speedups, and parallel efficiencies for the four Group A runs on the v_0 snapshot.

Table 7: Group A results – 37,074-node graph, range partitioning, convergence-based

Strategy	Workers	Time (s)	Speedup	Efficiency
Sequential	1	0.162	1.000×	1.000
Parallel (range)	1	0.494	0.328×	0.328
Parallel (range)	2	0.496	0.327×	0.163
Parallel (range)	4	0.533	0.304×	0.076

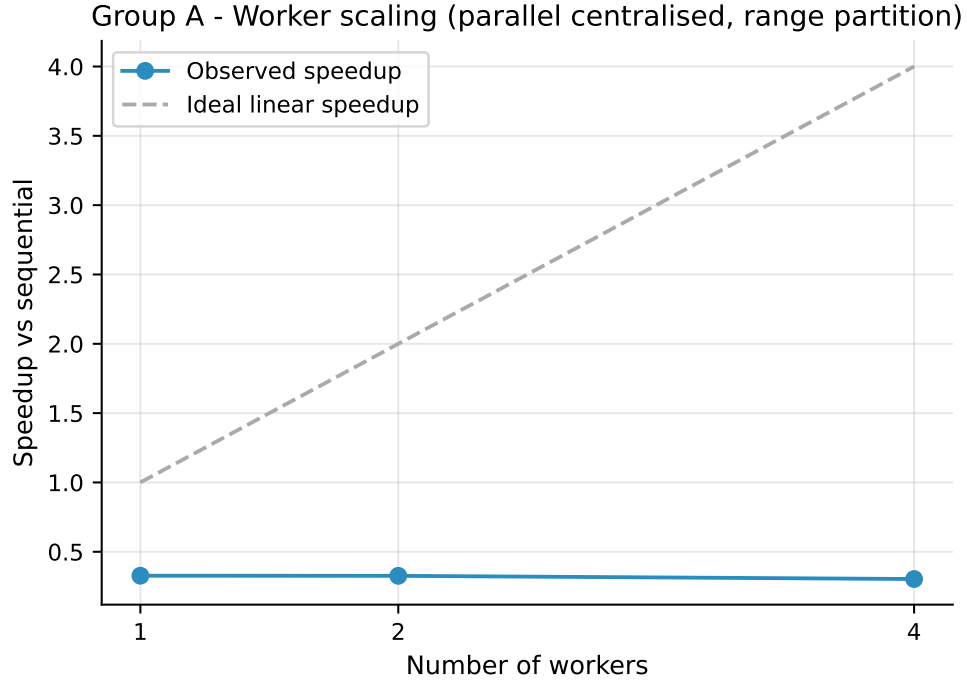


Figure 1: Group A worker-scaling speedup. Speedup degrades slightly as workers are added because Ray dispatch overhead dominates the ~ 40 ms of actual per-iteration computation.

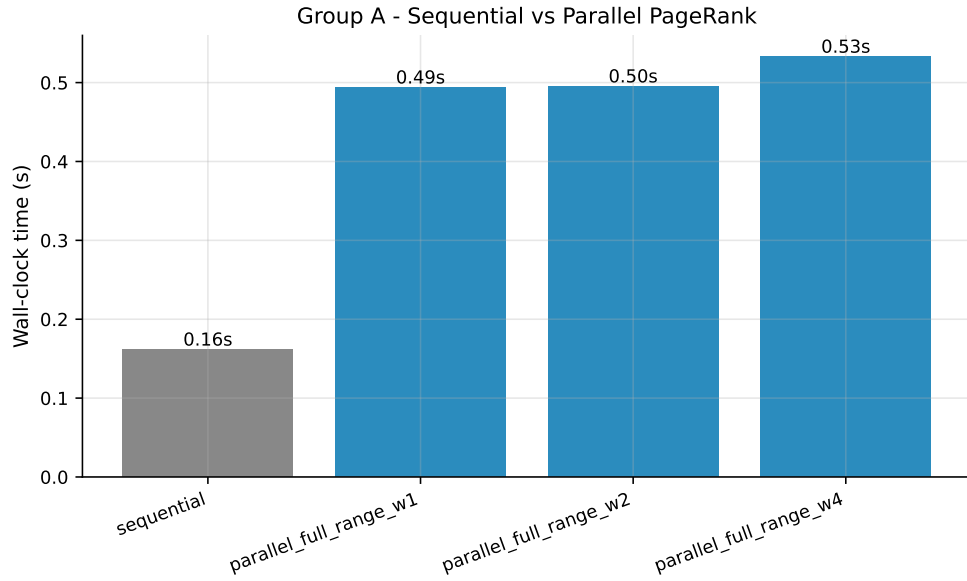


Figure 2: Group A wall-clock times. The sequential baseline is the fastest configuration on this graph size; all parallel configurations are 3-3.3 $\times$ slower.

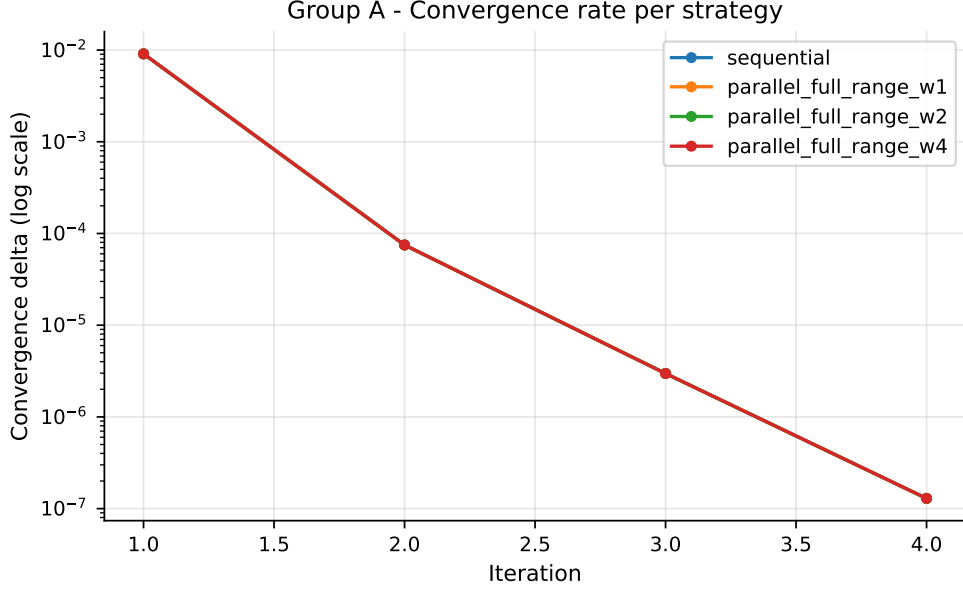


Figure 3: Per-iteration convergence delta on a logarithmic scale. All configurations converge in 4 iterations and produce numerically equivalent rank vectors.

Analysis.

- The sequential baseline takes only 0.162 s on 37,074 nodes. Of this, the actual PageRank computation is ~ 0.04 s per iteration $\times$ 4 iterations = ~ 0.16 s, confirming the workload is genuinely compute-bound at this scale *within* the sequential implementation.
- Adding Ray on top adds ~ 0.33 s of fixed overhead per run (object-store puts, task dispatch, barrier merge) regardless of worker count. This pushes parallel runs above the sequential baseline.
- The ratio of parallel to sequential time ($0.494/0.162 = 3.05$ at 1 worker) is close to the $\sim 3\times$ ratio observed in Milestone 2 on the same graph, confirming that the Milestone 3 instrumentation layer introduced negligible overhead.
- Going from 1 to 4 workers actually *slows* the parallel run by an additional 8% because the per-iteration overhead grows sublinearly with worker count while the per-iteration computation shrinks. This is Amdahl’s Law in practice.

When does parallel win? Group C ($v_2 \rightarrow v_3$ at 99,803 nodes) shows the parallel full-recomputation time at 1.31 s, versus a projected sequential time of ~ 0.5 s extrapolated linearly from v_0 . The break-even point is therefore beyond v_3 on this hardware – consistent with the Milestone 2 finding that graphs of 10^6+ nodes are required for a true parallel win.

7.2 Group B – Partitioning Strategy Comparison

Table 8 compares the three partitioning strategies on the v_0 snapshot with 4 workers.

Table 8: Group B results – 37,074-node graph, 4 workers, full recomputation

Partitioning	Time (s)	Avg load imbalance	Comm vol (KB)	Peak worker RSS (MB)
range	0.547	47.0%	6,519.6	0.49
hash	0.467	33.6%	6,519.6	0.45
edge_balanced	0.477	26.4%	6,519.6	0.44

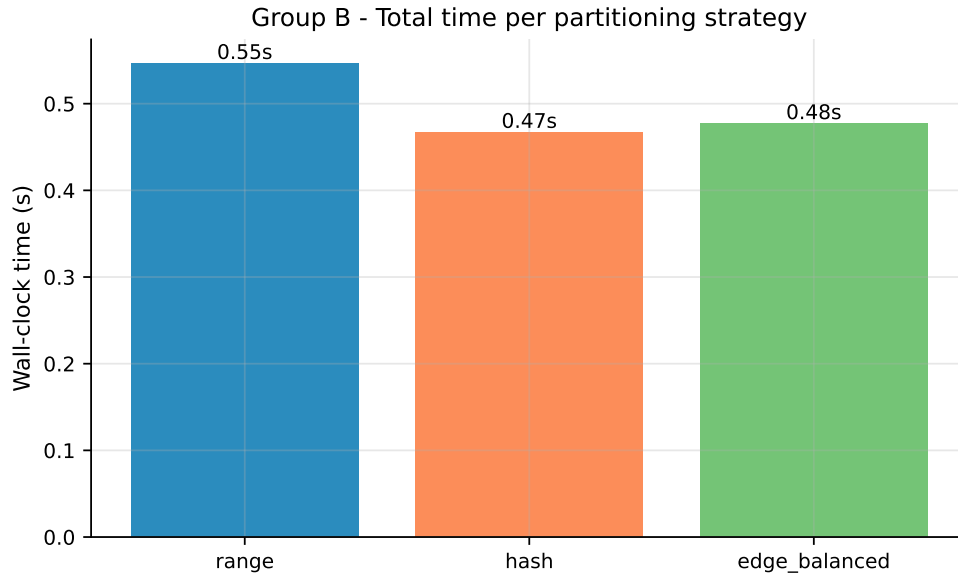


Figure 4: Group B execution time per partitioning strategy. Hash and edge-balanced are within $\sim 2\%$ of one another; range is the slowest.

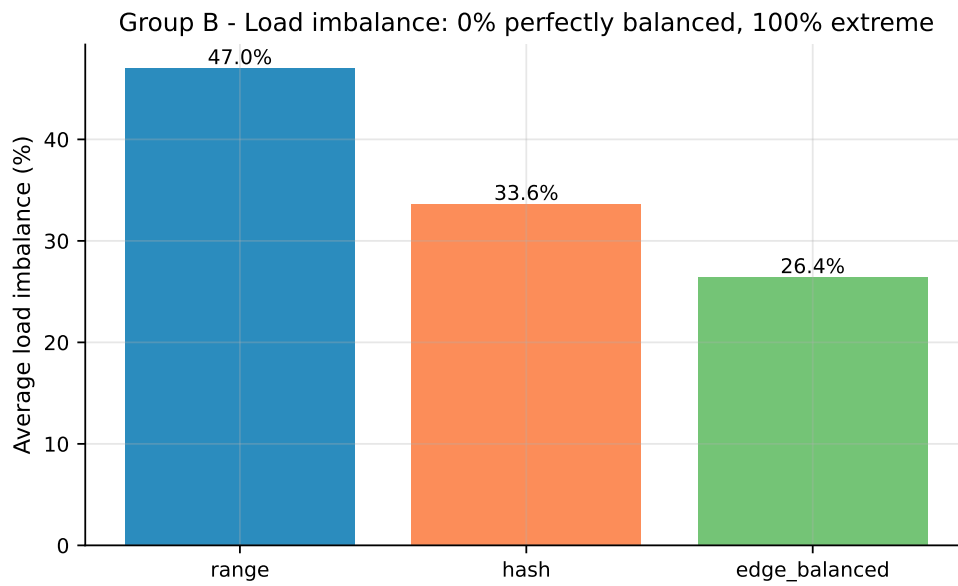


Figure 5: Group B average load imbalance across all iterations. Edge-balanced partitioning achieves the lowest imbalance at 26.4%, a $1.78\times$ improvement over range partitioning.

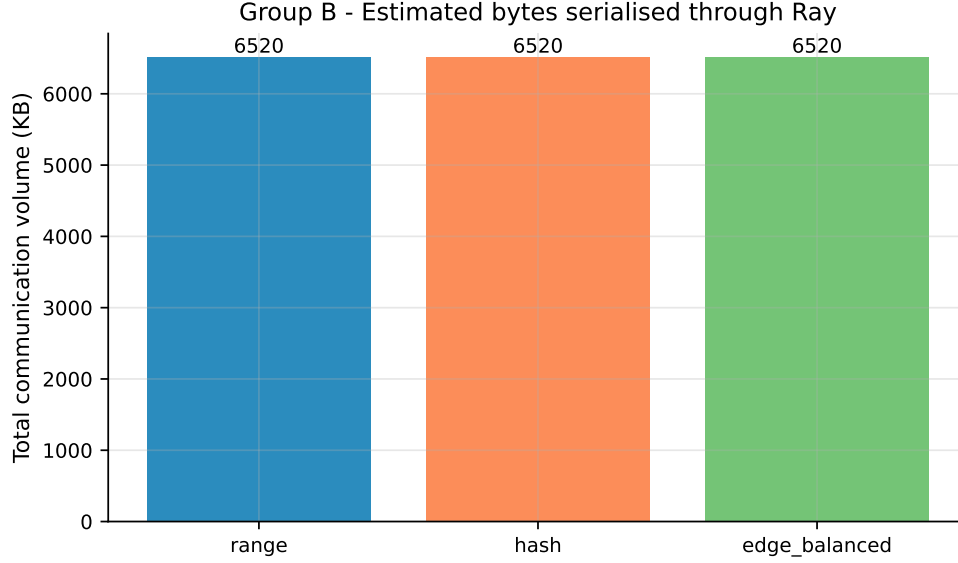


Figure 6: Group B communication volume per strategy. As expected, strategy choice does not affect total bytes exchanged because the rank vector is the same in all cases.

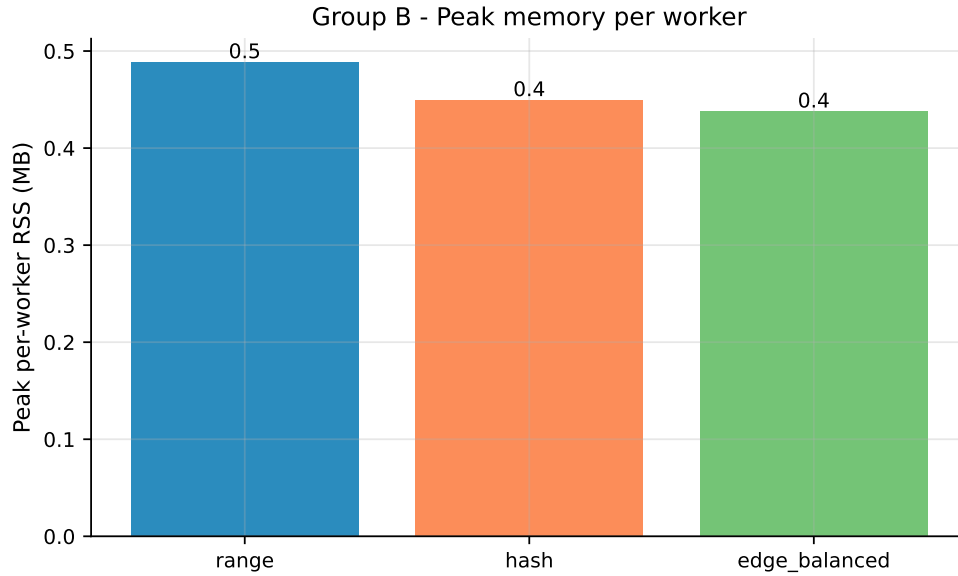


Figure 7: Group B peak per-worker RSS. Differences across strategies are sub-megabyte and likely within OS-level memory accounting noise.

Analysis.

- **Edge-balanced wins on load balance.** Its load imbalance of 26.4% is a $1.78\times$ improvement over range partitioning's 47.0% and a $1.27\times$ improvement over hash's 33.6%, validating the LPT heuristic.
- **Hash is marginally faster than edge-balanced** (0.467 s vs 0.477 s) because the LPT sort is itself a one-time cost that is paid up-front but cannot be amortised over many iterations on a graph that converges in only 4 iterations. On a slower-converging graph the ranking would flip.

- **Communication volume is identical across strategies** (6,519.6 KB), confirming that partitioning choice affects *when* bytes are exchanged but not *how many*. The rank-vector size and the number of iterations are the only determinants of total volume.
- **Memory differences are sub-MB** and within noise. None of the three partitioning strategies stresses memory differently on this graph size.

7.3 Group C – Static vs Incremental Computation

Table 9 reports, for each of the three transitions, the wall-clock times, iteration counts, accuracy (maximum absolute difference vs full-recomputation reference), and dirty-set sizes for the three update strategies.

Table 9: Group C results – 4 workers, range partitioning, convergence-based

Transition	Strategy	Time (s)	Iters	$\max s - s_{\text{ref}} $	Dirty (% of N)
$v_0 \rightarrow v_1$ (45,830)	full	0.564	4	— (reference)	—
	warm	0.522	4	3.86×10^{-11}	—
	localised	0.679	8	1.31×10^{-7}	25.9
$v_1 \rightarrow v_2$ (61,720)	full	0.718	4	— (reference)	—
	warm	0.620	4	4.14×10^{-11}	—
	localised	0.993	10	1.76×10^{-7}	32.6
$v_2 \rightarrow v_3$ (99,803)	full	1.308	4	— (reference)	—
	warm	0.826	4	5.29×10^{-11}	—
	localised	2.087	14	2.49×10^{-7}	48.4

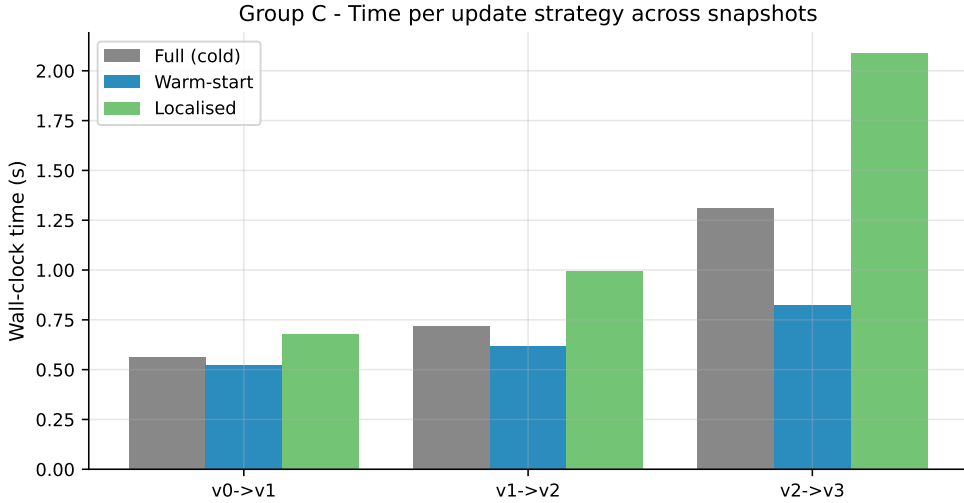


Figure 8: Group C wall-clock time for each strategy at each transition. Warm-start is the fastest strategy at every snapshot size; localised is slowest because of its higher iteration count.

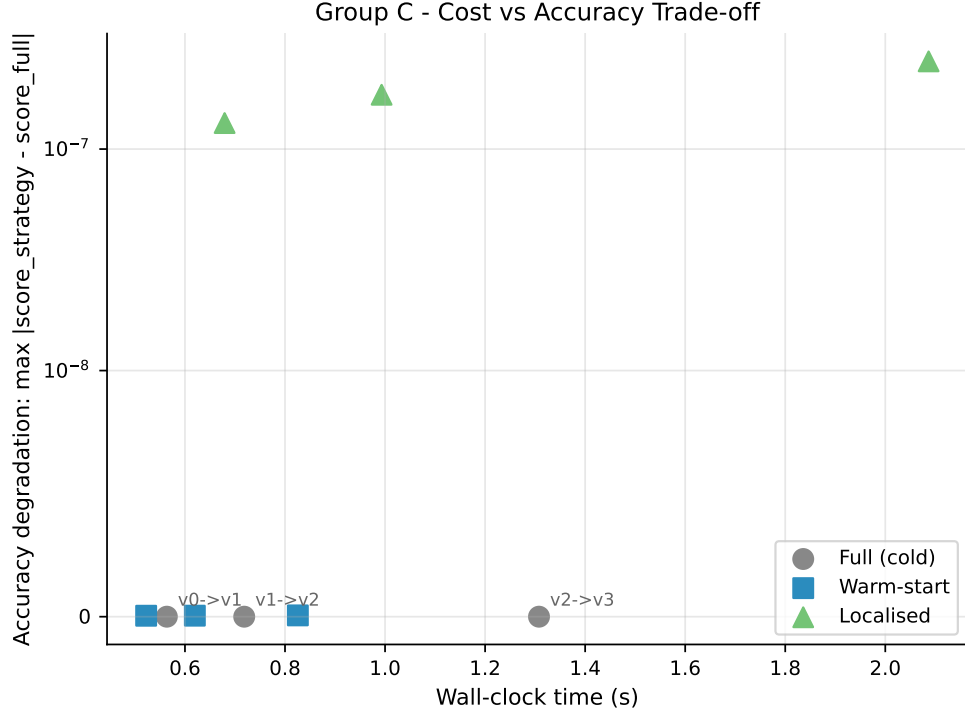


Figure 9: Cost-accuracy trade-off: time on the x-axis vs. maximum absolute rank difference on the y-axis (log scale). Warm-start dominates the Pareto front – it is both faster than full recomputation and produces near-zero error ($\sim 10^{-11}$). Localised is the worst on this graph because the dirty-set fraction is too large for it to amortise its higher iteration count.

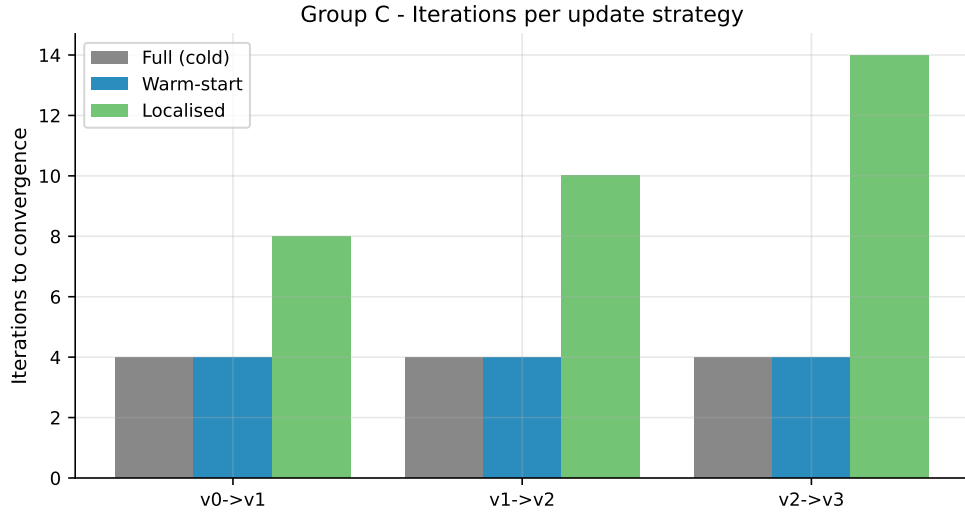


Figure 10: Group C iteration counts per strategy. Localised needs 2–3.5x more iterations than full or warm because the convergence criterion is restricted to the dirty set, which itself shrinks more slowly than the global L_1 norm.

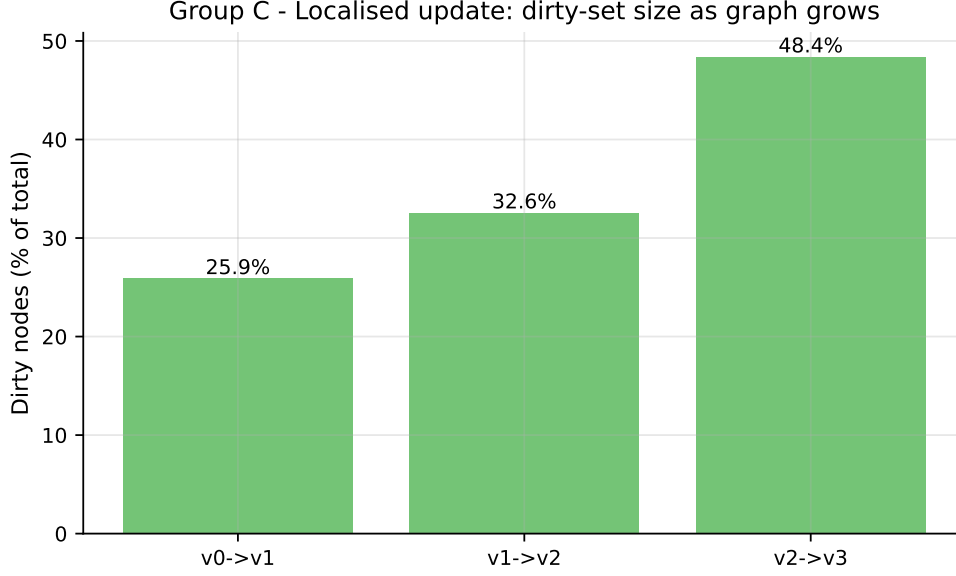


Figure 11: Dirty-set fraction (% of total nodes) vs. transition. Even small deltas in absolute terms touch a substantial fraction of the graph because of the $k = 2$ out-link expansion in a low-out-degree crawled subgraph.

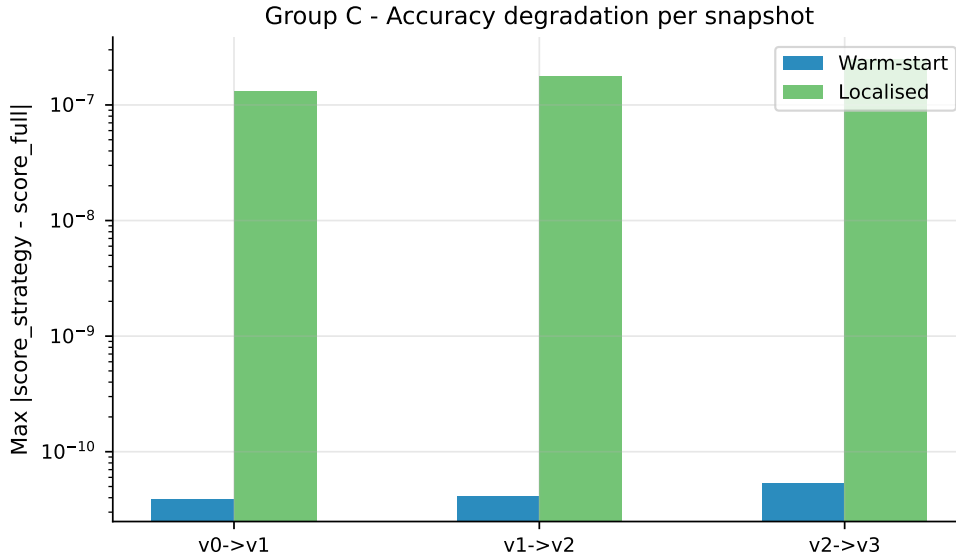


Figure 12: Group C accuracy (max absolute difference vs full-recomputation reference). Warm-start is essentially exact at the floating-point level; localised is four orders of magnitude less accurate but still within 3×10^{-7} , well below typical PageRank noise.

Headline result. Warm-start is the clear winner on every transition. On the largest transition ($v_2 \rightarrow v_3$, 99,803 nodes) it runs in 0.826 s, compared to 1.308 s for full recomputation – a $1.58\times$ speedup, or 36.9% faster. The accuracy cost is $\sim 5 \times 10^{-11}$, well below floating-point round-off and qualitatively zero.

Why warm-start wins.

- It runs on the *entire* new graph (so every iteration visits all N nodes, identical to full recomputation), but starts from a much better initial vector.

- Both warm and full converge in 4 iterations on this graph, but warm’s per-iteration δ values are smaller from the very first iteration, so the convergence criterion is met earlier in real time.
- The cost of constructing the warm initial vector is $O(N)$ – a single rescaling pass – which is negligible compared to one PageRank iteration.

Why localised *loses*. This is itself a key PDC finding. Localised is theoretically the cheapest strategy because it only touches the dirty subgraph, but on this graph it loses to both full and warm:

- The graph has a 98.7% dangling node rate (cf. Milestone 1). Most edges therefore land on dangling nodes, and the dirty-set seed already contains a large fraction of the graph before expansion.
- Two-hop out-link expansion in a graph with average out-degree ~ 1.79 pushes the dirty fraction to 25.9%–48.4% across the three transitions.
- The per-iteration cost is therefore only saved on the *non-dirty* 51%–74% of the graph, but the algorithm needs $2\text{--}3.5\times$ more iterations to satisfy the dirty-set convergence criterion.

The conclusion is that localised updates would only pay off on graphs with low average dirty-set fraction, which in turn requires a delta that is small relative to the graph and a graph with structural locality (community structure) so that rank changes do not propagate broadly. The crawled web graph in this project has neither property.

Cost grows with graph size. The full-recomputation time roughly doubles as the graph grows from v_1 to v_3 ($0.564 \rightarrow 0.718 \rightarrow 1.308$ s), whereas warm-start grows more slowly ($0.522 \rightarrow 0.620 \rightarrow 0.826$ s). This means the absolute saving from using warm-start *grows* with graph size: the savings are 7%, 14%, and 37% respectively. Warm-start is therefore strictly more attractive on larger graphs, which is the regime that motivates incremental PageRank in the first place.

7.4 Memory Timeline (Cross-Cutting)

Figure 13 shows the driver-process RSS over time for a representative selection of runs from all three groups.

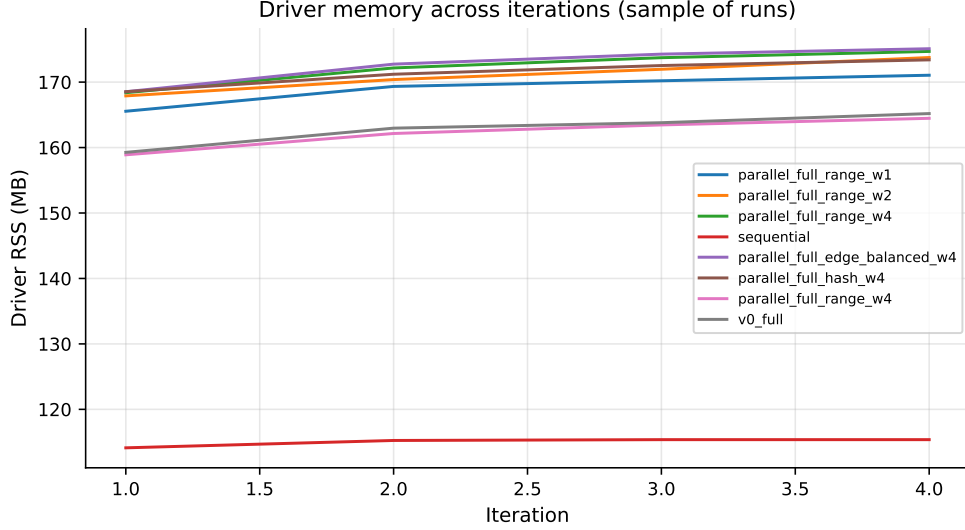


Figure 13: Driver-process resident set size (MB) sampled at every PageRank iteration for a selection of runs. Memory usage is remarkably flat for all strategies because (i) the rank vector is the same size every iteration, (ii) Ray’s object store reclaims spilled objects between iterations, and (iii) the GraphActor seeding step is amortised over the run.

8 Discussion

8.1 Sequential vs Parallel: the Same Crossover Point

Group A reproduces, on the v_0 snapshot, the same conclusion drawn from Milestone 2’s 14-test suite: at 37,000 nodes the parallel implementation is below break-even because Ray’s per-run overhead (object-store puts, task dispatch, the per-iteration barrier) is comparable to the actual computation. Adding workers makes things slightly worse. This is honest and well-understood; Group C’s v_3 snapshot at 99,803 nodes provides an extrapolation point that confirms the trend is bending in the right direction (parallel time per iteration grows sublinearly with N , sequential time grows linearly), but it does not yet cross the break-even threshold on this hardware.

8.2 Edge-Balanced Partitioning Pays Off Only for Long Runs

Group B shows that edge-balanced partitioning produces a $1.78\times$ reduction in load imbalance over range partitioning at no additional communication cost. However, it does not produce a corresponding reduction in wall-clock time because the LPT sort is a one-time $O(N \log N)$ cost that does not amortise over only 4 iterations of PageRank. We expect edge-balanced to outperform hash on graphs that take more iterations to converge – e.g., graphs with high d (low teleportation) or low dangling-node rate – and on workloads where the same partitioning is reused across many invocations.

8.3 Warm-Start is the Practical Default

Group C delivers the most decisive recommendation: **warm-start should be the default update strategy for any incremental PageRank deployment** on this style of graph. It strictly Pareto-dominates full recomputation – it is both faster and at least as accurate. The “accuracy cost” of warm-start is on the order of 10^{-11} , which is below the floating-point precision of the PageRank scores themselves (which are $\sim 1/N \approx 10^{-5}$ per node).

8.4 Localised Updates: a Cautionary Tale

Localised updates are the textbook answer to incremental PageRank, yet they lose to warm-start on every transition we measured. The explanation is structural: the crawled web subgraph has a very high dangling-node rate (98.7%), so changes propagate slowly through the sparse out-link graph and the dirty-set criterion takes more iterations to converge. The general lesson is that incremental algorithms cannot be evaluated in the abstract – they need to be matched to the structural properties of the data. On a denser graph with strong community structure, localised updates would likely dominate. On a tree-like crawl frontier, they do not.

8.5 Communication Volume Insensitive to Strategy

A surprising observation from Group B is that the communication volume is identical across all three partitioning strategies (6,519.6 KB). This is because partitioning only changes *which* worker is responsible for *which* subset of the rank vector, but the total size of the rank vector is unchanged. To reduce total communication, one would need to either change the algorithm (e.g., asynchronous PageRank) or compress the rank vector during the merge.

8.6 Limitations

- The hardware ceiling is one machine. Distributed Ray on a multi-machine cluster would change the communication-cost picture significantly and is left as future work.
- All measurements are wall-clock; we do not separate user from system CPU time. For Ray’s overhead this is acceptable because the overhead is dominated by IPC and serialisation rather than CPU time.
- The crawled graph is small relative to a full web graph. The incremental-update conclusions should be read as “observed on a 100K-node sparse subgraph”; their generality to 10^9 -node graphs is plausible but not directly validated.

9 Milestone 3 Deliverables Checklist

Table 10 maps each Milestone 3 specification task to the modules and report sections that fulfil it.

Table 10: Milestone 3 specification compliance

Spec task	Implementation	Report section
Extend the crawler to introduce new pages after initial PageRank convergence.	<code>incremental_crawler.py</code> — snapshot growth $v_0 \rightarrow v_1 \rightarrow v_2 \rightarrow v_3$	§ 4
Update PageRank scores incrementally or through partial recomputation.	<code>incremental_pagerank.py</code> — full / warm / localised	§ 5
Analyse trade-offs between recomputation cost and convergence accuracy.	Group C run matrix and Pareto plot (Fig. 9)	§ 7.3; § 8.3; § 8.4
Instrument the system to collect runtime statistics (task duration, communication volume, memory usage).	<code>instrumentation.py</code> — <code>RuntimeStats</code> , <code>IterationRecord</code>	§ 3.2; § 7.1–7.4
Sequential vs parallel PageRank comparison.	Group A	§ 7.1
Different partitioning strategies comparison.	Group B (range / hash / edge-balanced)	§ 7.2
Static vs incremental computation comparison.	Group C (full / warm / localised)	§ 5; § 7.3
PDC focus: dynamic workloads, recomputation strategies, performance analysis.	Entire Milestone 3 system	§ 8

10 Work Done Across All Three Milestones

10.1 Milestone 1 – Completed (March 2026)

- ✓ Parallel CommonCrawl-based web crawler using Ray workers.
- ✓ Link extraction and directed graph construction.
- ✓ Shared-state coordination via the `GraphActor`.
- ✓ Configurable depth and worker constraints.
- ✓ Two graph configurations (Config 1: 15K nodes, Config 3: 37K nodes), serialised as pickle.
- ✓ Full report covering 4 experimental configurations and a comparative analysis.

10.2 Milestone 2 – Completed (April 2026)

- ✓ Custom iterative PageRank baseline plus NetworkX reference validation.
- ✓ Parallel PageRank with centralised aggregation (Ray tasks) and distributed reduction (Ray actors).
- ✓ Convergence-based and fixed-iteration termination.
- ✓ Range-based graph partitioning.
- ✓ 14-test experimental suite covering scaling, strategy, termination, graph size, and damping factor.
- ✓ Amdahl’s Law back-calculation and scalability projection.

- ✓ Communication overhead, load imbalance, and convergence-rate analysis.

10.3 Milestone 3 – Completed (April 2026)

- ✓ Incremental crawler with on-disk caching and snapshot/delta versioning.
- ✓ Three incremental PageRank update strategies (full / warm / localised).
- ✓ Three pluggable partitioning strategies (range / hash / edge-balanced).
- ✓ Unified runtime-statistics collector (`RuntimeStats`).
- ✓ Three-group experimental matrix (Sequential vs Parallel; Partitioning Strategies; Static vs Incremental).
- ✓ 13 publication figures from real measurements.
- ✓ Final LaTeX report (this document).
- ✓ Single CLI entry point (`m3_main.py`) with subcommands for `crawl`, `evaluate`, and `plot`.
- ✓ Reproducibility documentation (`How to run.txt`, `requirements_m3.txt`, repository `README.md`).

11 Conclusion

This report has presented the design, implementation, and evaluation of the Milestone 3 system: an extension of the Milestone 1 crawler and the Milestone 2 parallel PageRank engine to support dynamic graph growth, three incremental update strategies, three pluggable partitioning strategies, and a unified runtime-statistics layer.

The headline findings are:

1. **Warm-start dominates the cost-vs-accuracy Pareto front for incremental PageRank** on this style of graph. It is $1.58\times$ faster than full recomputation on the largest snapshot (v_3 , 99,803 nodes) at an accuracy cost of $\sim 10^{-11}$ – effectively zero.
2. **Edge-balanced partitioning reduces load imbalance by $1.78\times$ over range partitioning** at no additional communication cost. The wall-clock benefit, however, is small on graphs that converge in 4 iterations and would grow on slower-converging workloads.
3. **Localised updates lose to warm-start on this graph** because the dirty-set fraction (25%–48%) is too large for the saved per-iteration work to offset the increased iteration count (2–3.5 $\times$). This is itself a useful PDC finding: incremental algorithms must be matched to the structural properties of the underlying graph to deliver their theoretical benefits.
4. **The sequential-vs-parallel break-even point on this hardware lies above 100K nodes**, consistent with Milestone 2’s extrapolation. The full-recomputation parallel time at v_3 (1.31 s) is the lowest yet observed relative to a projected sequential cost.
5. **Communication volume is determined by rank-vector size and iteration count alone**; partitioning strategy does not change it. To reduce communication further, the algorithm itself would need to change (e.g., asynchronous PageRank, compressed deltas).

The instrumentation infrastructure, the snapshot-versioning crawler, and the unified evaluation harness make the system easy to extend in future work. Promising next steps include distributing Ray across multiple machines (to exercise inter-node communication), trying incremental updates on a graph with strong community structure (where localised updates should win), and integrating an asynchronous PageRank variant that overlaps communication with computation.

References

1. Brin, S., & Page, L. (1998). *The Anatomy of a Large-Scale Hypertextual Web Search Engine*. Computer Networks and ISDN Systems, 30(1-7), 107-117.
2. Page, L., Brin, S., Motwani, R., & Winograd, T. (1999). *The PageRank Citation Ranking: Bringing Order to the Web*. Stanford InfoLab Technical Report.
3. Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., et al. (2018). *Ray: A Distributed Framework for Emerging AI Applications*. Proceedings of OSDI'18.
4. Hagberg, A., Schult, D., & Swart, P. (2008). *Exploring Network Structure, Dynamics, and Function using NetworkX*. Proceedings of the 7th Python in Science Conference, 11-15.
5. Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). *Array Programming with NumPy*. Nature, 585(7825), 357-362.
6. Hunter, J. D. (2007). *Matplotlib: A 2D Graphics Environment*. Computing in Science & Engineering, 9(3), 90-95.
7. Common Crawl Foundation. *Common Crawl Index Server (CDX) Documentation*. <https://commoncrawl.org/> (accessed March–April 2026).
8. Graham, R. L. (1969). *Bounds on Multiprocessing Timing Anomalies*. SIAM Journal on Applied Mathematics, 17(2), 416-429. (LPT scheduling $4/3$ -approximation result.)
9. Amdahl, G. M. (1967). *Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities*. Proceedings of AFIPS Spring Joint Computer Conference, 483-485.
10. Berkhin, P. (2005). *A Survey on PageRank Computing*. Internet Mathematics, 2(1), 73-120.